docuflow

Parallel ingest and agentic RAG over scanned documents that mix printed and handwritten text

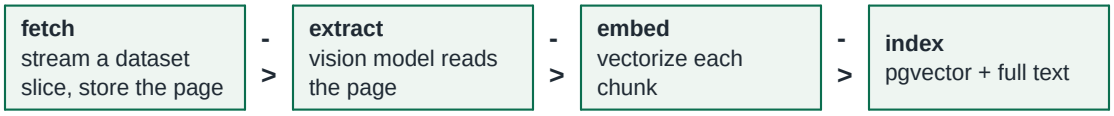
This document explains what was built, how the pieces fit together, what was measured, and what is still untested. Everything described is in `llmd-project/docuflow`.

1. The goal

Build a system around four public datasets that contain both printed and handwritten content, where documents can be **ingested in parallel while users query what has already been indexed**, the workload is managed (priorities, limits, scaling), and questions are answered by an **agentic RAG** loop rather than a single retrieve-then-generate call.

Dataset	Printed	Handwritten	Used here for
FUNSD	form labels, boilerplate	filled-in answers, signatures	forms; page-reading accuracy
XFUND	forms in 7 languages	answers, names, numbers	multilingual extraction
DocVQA	memos, letters, tables	approvals, margin notes	question answering evals
IAM	printed prompts (full forms)	transcription lines	handwriting accuracy. The Hub copy has line images only, so no printed prompts.

2. How it works



Each stage is a job in a Postgres queue. A page is enqueued for extraction the moment it is fetched, and becomes searchable the moment it is embedded. That is what lets queries run *during* ingestion: they see whatever is indexed so far, and the agent is told the corpus may still be filling.

Page reading

A vision model returns every text region with a **kind** (printed or handwritten) and a **role** (header, label, value, note, signature...). Each chunk line keeps its tag, for example `(handwritten/value) J. Smith`, so the agent can filter on it and cite it.

Storage

One Postgres database holds documents, chunks (pgvector embeddings plus a full-text index), raw extractions, the job queue, and query logs.

3. Workload management

Rules are applied when a worker claims a job, and most can be changed live without a restart (CLI, API, or the dashboard):

Control	What it does
Lanes	io (fetching), vlm (page reading), embed. Each has its own worker pool and concurrency limit across all processes.
Dataset cap + fair share	No dataset can hold all vlm slots; ties go to the dataset with the fewest running jobs.
Priority	Later stages outrank earlier ones, so pages finish and become queryable sooner; per-ingest priority is settable.
Pause / resume	Pause a lane or a dataset; running jobs finish, nothing new starts.
Query yield	While interactive questions are in flight, the vlm lane starts no new pages, so queries are not stuck behind page reading.
Failure handling	Retries with exponential backoff, a dead-letter state with *retry-dead*, lease expiry for crashed workers, orphan recovery at startup, and a supervisor that restarts dead processes.

4. Agentic RAG

A small LLM runs a loop. On each turn it returns one JSON action and sees the result, until it answers with citations:

Action	Purpose
search	hybrid search (vector + full text, rank fusion), optional dataset and printed/handwritten filters
get_page	full extracted text of one page
read_page_image	have the vision model look at the page image to answer a specific question (unclear handwriting, missing text)
corpus_stats	what is indexed and what is still pending
answer	final answer with citations to pages the agent actually saw

It must search before it may answer (small models otherwise give up early), it can rewrite a failed query, and the full step-by-step trace is returned with every answer for debugging.

5. What was measured

Local run on one Mac: 30 pages (IAM 12, FUNSD 6, XFUND 6, DocVQA 6), gemma4 vision model, qwen3 4B agent, qwen3 embeddings. All 30 pages indexed, zero failed jobs.

Dataset	Pages	Char. error	Word overlap (F1)	Seconds / page
IAM (handwriting)	12	0.02	0.91	3.7
FUNSD	6	0.17	0.87	33
XFUND (multilingual)	6	0.37	0.86	40

Query latency	Queries	Mean	Median
Idle	6	26.5 s	26.3 s
During ingestion	3	48.5 s	46.9 s

Read these numbers carefully. DocVQA answers: 2 of 6 correct on a strict match, which is too small a sample to conclude anything (one miss was only a formatting difference, another was a real error). Latency is high because three models share one machine. The printed/handwritten label is produced by the model and is **not verified**: no dataset provides ground truth for it, and one handwritten IAM line was labelled printed.

6. Running it on Kubernetes (fake GPUs)

Plain Kustomize manifests (no Helm) on the existing kind cluster with 6 fake GPUs, KEDA and the tenant priority classes:

Component	Scaling
worker-vlm (requests 1 fake GPU, lowest priority)	KEDA Postgres scaler on queued + running jobs: 0 to 6 replicas, scales to zero when idle
worker-io, worker-embed	KEDA, 1 to 2 replicas
api	CPU-based HPA, 1 to 3 replicas

Result of the demo burst (60 pages): KEDA took vlm workers from 0 to 1 within about 17 seconds, then asked for 4 and 6. Only 3 fake GPUs were free next to the other tenants, so 3 pods ran and 3 stayed Pending until the queue drained, then everything scaled back to 0. All 60 pages were indexed. A simulated model service (fixed latency, synthetic output) stood in for the models, so this demonstrates scheduling and scaling, not extraction quality.

Problems found and fixed on the way: the KEDA operator could not resolve the short database name (now a full service name); the io worker was killed for memory on large XFUND scans (limit raised to 3 GiB).

7. Production on real GPUs (AKS)

A separate overlay (`k8s/overlays/gpu`) targets the AKS cluster from the earlier llm-d work:

- **GPUs belong to the vLLM model servers**; docuflow workers are clients and request no GPU. The vision server scales 1 to 3 on in-flight requests.
- **All model calls go through LiteLLM** (vision, agent, embedding aliases): per-tenant key, budget and rate limit. A new OpenAI-compatible backend speaks to vLLM, with retries on 429/5xx.

- **Hardening:** secrets for DSN and keys, API-key authentication, non-root pods with read-only filesystem, disruption budgets, a network policy around Postgres, probes.
- **Observability:** a `/metrics` endpoint (queue depth and age per lane, documents by status, query latency) and a `/ready` check.

Status: not yet run on a real GPU cluster. No Azure CLI or subscription was available. Verified: the overlay renders and passes a server-side dry run; the new backend ran against real local models and against the simulator; the scaling logic ran on the fake-GPU cluster. Expect to tune vLLM flags, worker counts and scaling thresholds on first contact with real hardware.

8. Known limits and next steps

- No real-GPU deployment yet; first deploy will need tuning.
- Eval sample is small (6 DocVQA questions); extraction quality of the production vision model (Qwen2.5-VL) is unmeasured.
- Page images are stored in Postgres: fine to hundreds of thousands of pages, then move them to object storage.
- Use a managed Postgres with pgvector in production instead of the in-cluster one.
- Preemption by higher-priority tenants is untested (it could evict another tenant's pod on the shared cluster).
- Authentication is a single shared API key; there is no per-user access control.

9. Using it

```
make setup db models
make up # API :8791 + workers
make ingest-all N=20
make ask Q="Who is the fax addressed to?"
python -m docuflow pause datasets xfund
make eval QA=12 | make test
make k8s-up && make k8s-ui && make k8s-demo
make gpu-docuflow-image ACR=... && make gpu-docuflow-up ACR=...
```

Code map: `docuflow/jobs.py` (queue and workload rules), `pipeline.py` (stages), `agent.py` (agentic RAG), `retrieval.py` (hybrid search), `llm.py` (model backends), `api.py`, `k8s/`, `evals/`.